22 Vlákna, Paralelní programování, Asynchroní metody, Concurrent patterns

Co je to vlákno?

Vlákno je v Javě jedna instance třídy Thread. Třída Thread umožňuje vykonával kód paralerně – to znamená, že dvě části programu mohou běžet najednou. Kdykoliv spustíme nějaké nové vlákno, JVM pro něj vytvoří vlastní samostatný zásobník na Stacku. Díky tomu může celé vlákno pracovat bezpečně s vlastními daty, aniž by nedocházelo k přepisování dat nějakého jiného vlákna.

Vlákna se JVM snaží různě dělit i mezi procesory, ale není to tak, že čtyři vlákna budou 100% na čtyřech procesorech. Je to ale hodně pravděpodobné. JVM multithreading umí.

Vlákna se v Javě dají implementovat dvěma způsoby.

Využití extendu

Jako první můžeme třídu podědit od třídy Thread. Poté přepíšeme a overridneme metodu run. Při spuštění se tedy vytvoří vlastní vlákno, na kterém budou probíhat věci, které jsou napsané v run metodě

Main.java

```java
package maturita;
public class Main extends Thread {
    public void run() {
        System.out.println("This code is running in a thread"); // Corrected quotation marks
    }

    public static void main(String[] args) {
        Main thread = new Main(); // Create an instance of Main
        thread.start(); // Start the thread
    }
}
```

Implementováním

Jako druhé můžeme třídu nechat implementovat Runnable. Poté přepíšeme a overridneme metodu run. Při spuštění se tedy vytvoří vlastní vlákno, na kterém budou probíhat věci, které jsou napsané v run metod. Také můžete vidět, že můžeme dát díky Runnable na vstup Threadu v Mainu třídu Task.

*Main.java *Task.java

```java
package maturita;

public class Task implements Runnable {
    private String taskName;

    public Task(String name) {
        this.taskName = name;
    }

    public void run() {
        System.out.println("Task " + taskName + " is running in thread " + Thread.currentThread().getName());
    }
}
```

Main.java

```java
package maturita;
public class Main extends Thread {
    public void run() {
        System.out.println("This code is running in a thread"); // Corrected quotation marks
    }

    public static void main(String[] args) {
        Main thread = new Main(); // Create an instance of Main
        thread.start(); // Start the thread
    }
}
```

Paralelní programování

Paralelní programování je způsob, jak psát programy paralelně. Je opakem sekvenčního, které instrukce zpracovává postupně jednu po druhé. Paralelizace se využívá v případě, že máme nějaký algoritums na řešení šíleného a náročného problému. My jej rozdělíme na několik různých částí, které každé budou najeno počítač a pak společně problém vyřeší rychleji. Díky tomu nějaké části kódu nemusejí čekat na něco, co jim stejně nepředá data.

Jelikož se ovšem vlákna spouštějí najednou, vede to někdy v problémům, kdy program vrátí pokaždé jiný výsledek. Takovým příkladem může být takovýto kód:

*Main.java *Task.java

```java
package maturita;
public class Main extends Thread {
    public int amount = 0;

    public static void main(String[] args) {
        Main thread = new Main();
        thread.start();
        System.out.println(thread.currentThread().getName());
        System.out.println(thread.amount);
        thread.amount++;
        System.out.println(thread.amount);
    }
    public void run() {
        System.out.println(Thread.currentThread().getName());
        amount++;
    }
}
```

V tomto kusu kódu se totiž vytvoří druhé vlákno od Mainu **(Main thread = new Main())**

První vlákno je samotný main. Toto druhé vlákno spustíme (pomocí **.start()** metody, což zavolá metodu **run()** a díky tomu se začnou tyto instrukce provádět na novém vlákně.

Výstupy se budou lišit, protože je proměnná amount sdílená mezi vlákny!

A výstupy… jsou opravdu různé.

Asynchronní metody

Asynchronní metody jsou takové metody, které neblokují svým spuštěním chod aplikace. Příkladem může být třeba spojení s databázovým serverem – Pokud máme slabé internetové spojení a bude komunikace a spojení trvat dlouho, třeba tři vteřiny, asynchronní metoda se bude s databázovým serverem spojovat tři vteřiny, ale program nebude tři vteřiny čekat, ale rovnou bude pokračovat v běhu a načítání různých dalších komponent.

Jakmile je asynchronní metoda dokončena, program pokračuje od toho řádku, kde byl využit join().

Cokoliv před Joinem tedy pokračuje a jede. Příklad:

*Main.java *Task.java

```
1  package maturita;
2
3  import java.util.concurrent.CompletableFuture;
4
5  public class Main {
6      public static void main(String[] args) {
7          CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {
8              System.out.println("Začínám asynchroní metodu!");
9              try {
10                 Thread.sleep(3000);
11                 System.out.println("PRÁVĚ JSEM DOKONČIL ASYNCHRONÍ METODU!");
12             } catch (InterruptedException e) {
13                 e.printStackTrace();
14             }
15         });
16         System.out.println("Metoda main pokračuje ve svém běhu. Tento kód je před joinem");
17         future.join();
18         System.out.println("Tento kód je po joinu");
19     }
20 }
21
```

V tomto příkladě my spustíme asynchronní metodu. Metoda main ale bude pokračovat dál a dál, dokud nebude future.join(). Ten čeká na asynchronní metodu a dokud není hotová, tak on dál nebude pokračovat. Jakmile se asynchronní metoda dokončí, kód bude pokračovat.

Výsledek by mohl být takový:

METODA MAIN POKRAČUJE VE SVÉM BĚHU. TENTO KÓD JE PŘED JOINEM

ZAČÍNÁM ASYNCHRONÍ METODU!

PRÁVĚ JSEM DOKONČIL ASYNCHRONÍ METODU!

TENTO KÓD JE PO JOINU

Ještě si můžete říct: Proč nejdříve main pokračuje a teprve poté se začíná s asynchronní metodou? Je to proto, že knihovna CompeatableFuture hodí asynchronní metodu do jiného vlákna. Pozor, asynchronní metody nemusí vždy být v jiném vlákně!

Úžasná věcička, že?

Concurrent design patterns

Jedná se o vzory, které řeší problémy, při kterých program běží na několika vláknech najednou a řeší souběžně určité operace. Mezi nejznámější patří například

• Thread Pool – Bazének několika vláken, které řeší určité problémy seřazené ve frontě.

• Read Write Lock – Tzv. RWL slouží k umožnění vláknům k přístupu pro čtení v jeden a ten samý čas.

Thread Pool

Thread pool je bazének vláken, které řeší různé problémy seřazené ve frontě a hlavní výhodou tohoto návrhového vzoru je dosažení konkurence – Stavu, ve kterém se výpočty a části programů mohou řešit mimo normální stanovený průběh.

Read Write Lock

Read Write Lock, také zvaný RWL umožňuje přístup k jednotlivým proměnným nebo polím v jeden moment. Write je exkluzivní, musí se použít lock. Jedná se také o synchronizační primitivum. Zámky fungují na tom principu, že jedno vlákno přistoupí k metodě. Ta na prvním řádku spustí lock -> V tu chvíli do ní nemůže jiné vláknou vstoupit a musí počkat, až první vlákno dojede. Teprve poté se zámek odemkne, druhé vlákno může vstoupit do metody.

Locky zabraňují možnostem v jednom momentu zapsat na stejnou pozici item, přistoupit k něčemu co

by už nemuselo existovat a podobné.